

1 Appendices

A Example Task Flow for Agentic Framework

User Task Prompt (T)

-prompt "Write a C program to compute the factorial of a number using recursion."

System Initialization

Programmer Agent (A_P)

System Prompt:

"You are a C programmer. You respond with the code in C to solve the task. No comments or explanations."

Designer Agent (A_d)

System Prompt:

"You are a C designer. You will be given a task and you will respond with design suggestions to solve the task."

These system prompts are inserted once during agent initialization and persist via internal memory ('self.messages') at both Designer and Programmer end.

► Execution Flow

→ Step 1: Programmer Receives Task

Prompt to A_P

- **(system)** You are a C programmer...
- **(user)** Write a C program to compute the factorial using recursion.

Response P_0

```
1 int factorial(int n) {
2     if (n <= 1) return 1;
3     else return n * factorial(n - 1);
4 }
5
6 int main() {
7     int n = 5;
8     printf("%d", factorial(n));
9     return 0;
10 }
```

✂ Step 2: Compiler Agent Validation

Compiler Agent A_C Process

- (1) Check if code is in Markdown; if so, extract C block.
- (2) Write extracted code to temp.c.
- (3) Attempt compilation via gcc.
- (4) **If successful:** return original code.
- (5) **If failed:**
 - Construct prompt to LLM:
"For this program <code>, I got the following compilation error: <error>. Please fix the code and return the fixed code in a markdown code block."
 - Retry up to TRIALS times (default: 3).
 - Return first fix that compiles, or last attempt if all fail as P'_0 .

➔ Step 3: Designer Receives Code

Prompt to A_d

- **(system)** You are a C designer...
- **(user)** Here is my program, how can I improve it?
<code from P'_0 >

Response D_0

Consider taking input from the user instead of hardcoding the value of n. Also, validate that the input is non-negative.

↻ Iteration 1

➔ A_P receives D_0

Prompt to A_P

- (system)** You are a C programmer...
- (user)** Write a C program to compute the factorial using recursion.
- Assistant:** <code from P'_0 >
- (user)** D_0 .

Response P_1

```

1 int main() {
2     int n;
3     printf("Enter a number: ");
4     scanf("%d", &n);
5     printf("%d", factorial(n));
6     return 0;
7 }

```

✂ Compiler Agent Validation - P'_1 **→ Designer Receives Code****Prompt to A_d**

(system) You are a C designer...

(user) Here is my program, how can I improve it?

<code from P'_0 >

(assistant) <design feedback D_0 >

(user) How to improve this code:

<code from P'_1 >

Response D_1

Add error handling for invalid input. Display a message if a negative number is entered.

↻ Iteration 2 +

Subsequent exchanges proceed without extra instruction wrappers:

Prompt to A_P (iteration 2)

(system) You are a C programmer...

(user) Write a C program to compute the factorial of a number using recursion.

(assistant) <code from P'_0 >

(user) <design feedback D_0 >.

(assistant) <code from P'_1 >

(user) <design feedback D_1 >.

Response P_2

```

1 int main() {
2     int n;
3     printf("Enter a number: ");
4     scanf("%d", &n);
5     if (n < 0) {
6         printf("Invalid input");
7         return 1;
8     }
9     printf("%d", factorial(n));
10    return 0;
11 }

```

✂ Compiler Agent Validation - P'_2

➔ Designer Receives Code

Prompt to A_d (iteration 2)

(system) You are a C designer...

(user) Here is my program, how can I improve it?

<code from P'_0 >

(assistant) <design feedback D_0 >

(user) How to improve this code:

<code from P'_1 >

(assistant) <design feedback D_1 >

(user) <code from P'_2 > (only the code this time)

Response D_2

Add comments to make your code more understandable. Also consider modularizing the input validation into a separate function for better readability and reusability.

🔄 Iterative Continuation (Step 3 to Step n)

From this point, the loop continues with the same structure:

- Programmer A_P receives design feedback D_{i-1} and generates updated code P_i
- Compiler Agent validates and corrects the code as P'_i if needed
- Designer A_d receives P'_i and responds with new feedback D_i

This cycle is repeated deterministically for up to $n = 100$ iterations.

 Save the Results

All messages up to this point are logged in the final Excel file.

B Programming Task Prompts

Table 1. List of Unsuccessful and Successful Task Prompts

Unsuccessful Prompt	Issue Observed	Successful Prompt
Write a game that uses Fibonacci numbers	Returned the Fibonacci function without any game structure or generated educational quizzes or explanations instead of playable logic	Write a mathematical game of Fibonacci
Develop a encryption-decryption game using Caesar Cipher	Returned only the encryption-decryption program using Caesar Cipher. No game logic	Develop a game using Caesar Cipher
Write a mathematical game of Caesar Cipher		
Develop a game of encryption-decryption using Caesar Cipher		
Given an integer x, return true if x is a palindrome, and false otherwise.	Returned only the palindrome checking function without input/output logic or a main function	Write a complete program that checks whether a given integer is a palindrome.
Given an array of integers, nums and an integer target, return indices of the two numbers in the array such that they add up to target. You may assume that each input would have exactly one solution, and you may not use the same element twice. You can return the answer in any order.	Returned only core function, no full program	Write a complete program to return indices of the two numbers in an integer array such that they add up to a given target. You may assume that each input would have exactly one solution, and you may not use the same element twice. You can return the answer in any order.

C Task Prototype Vector

Table 2. Task Prototype Definitions Used for Topic Stability Analysis

Problem	Task Prototype Description
Fibonacci Game	This task involves writing and improving a mathematical Fibonacci game in C. It should involve Fibonacci number calculation and a gameplay loop, including rounds, scoring, user interaction, and quiz mechanics.
Caesar Cipher Game	This task involves writing a program that should include Caesar Cipher logic, user-provided shift values, menu-driven interaction, gameplay rounds, difficulty levels and scoring
Temperature Cleaning Tool	This task involves writing and improving a temperature data cleaning program in C. It should involve detecting missing, duplicated, or unrealistic temperature values (e.g., below -100°C or above 70°C), along with logic to correct or flag invalid entries and handle structured numerical input.
Anagram Game	This task involves writing and improving an anagram-based word game in C. It should involve string manipulation, character comparison logic, user interaction, validation of whether two words or phrases form anagrams, gameplay rounds, difficulty levels and scoring.
Palindrome Check	This task involves writing and improving a palindrome-checking program in C. It should involve validating whether a given integer is a palindrome using digit extraction or comparison logic, along with basic input handling and conditional checks.
Array Index Sum	This task involves writing and improving a C program to identify two indices in an array whose elements sum to a given target value. It should involve array traversal, constraint handling to avoid reuse of the same element, and logic to return the indices in any valid order.
Linked List Number Addition	This task involves writing and improving a C program to add two non-negative integers represented as reversed linked lists. It should involve linked list traversal, digit-wise addition with carry handling, and construction of a resulting linked list in reverse order.
Median of Two Sorted Arrays	This task involves writing and improving a C program to compute the median of two sorted arrays. It should involve efficient array partitioning or divide-and-conquer logic to achieve logarithmic time complexity, along with careful boundary and index handling.

D Model Snapshot Information

Table 3. Model snapshot information for all Ollama-hosted models used in the study (snapshot month: December 2025).

Paper Label	Ollama Tag	Model's Digest	Developer	Params	General Architecture	Quantization
DeepSeek-R1 (7B)	deepseek-r1:7b	755ced02ce7b	DeepSeek	7.62B	qwen2	Q4_K_M
DeepSeek-R1 (32B)	deepseek-r1:32b	edba8017331d	DeepSeek	32.8B	qwen2	Q4_K_M
DeepSeek-R1 (70B)	deepseek-r1:70b	d37b54d01a76	DeepSeek	70.6B	llama	Q4_K_M
DeepSeekCoder (33B)	deepseek-coder:33b	acec7c0b0fd9	DeepSeek	33.3B	llama	Q4_0
Qwen3Coder (30B)	qwen3-coder:30b	06c1097efce0	Qwen / Alibaba	30.5B	qwen3moe	Q4_K_M
CodeLlama (13B)	codellama:13b	9f438cb9cd58	Meta	13B	llama	Q4_0
CodeLlama (34B)	codellama:34b	685be00e1532	Meta	33.7B	llama	Q4_0
Qwen3 (30B)	qwen3:30b	ad815644918f	Qwen / Alibaba	30.5B	qwen3moe	Q4_K_M
Llama 3.3 (70B)	llama3.3:70b	a6eb4748fd29	Meta	70.6B	llama	Q4_K_M